JAILED3
Security Review



MAY, 2026

www.chaindefenders.xyz
https://x.com/DefendersAudits

Lead Auditors



PeterSR



0x539.eth

Table of Contents

1. Protocol Summary
2. Disclaimer
3. Risk Classification
4. Audit Details

- Commit Hash
- Scope
- Roles

5. Executive Summary

- Issues Found
- Issues Statuses
- Security Grade

6. Findings

- High
- Medium
- Low
- Gas

Protocol Summary

Jailed3 is a Solana-based on-chain game program built with Anchor. It implements a token-farming game with breeding mechanics where players join the game, manage farms to earn $JAIL tokens, breed prisoner assets, and trade on a fixed-price marketplace. The protocol features a Token-2022 mint with transfer fees, Ed25519 signature authority gating for core actions, MagicBlock VRF integration for breeding and pack-opening randomness, a daily emission schedule

with configurable rates, and 36 instructions spanning admin configuration, farming, breeding/-pack systems, and marketplace operations. The program includes a shutdown recovery framework and full governance controls for game parameters including marketplace fees, farm upgrade costs, and player rewards.

Disclaimer

The Chain Defenders team makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the implementation of the contracts.

Risk Classification

Impact/Likelihood	High	Medium	Low
High	High	High	Medium
Medium	High	Medium	Low
Low	Medium	Low	Low

Audit Details

Commit Hash

8ce66ad4740bd9263d43a00f54fe6595a83f54ca

Scope

Id	Files in scope
1	programs/game/src/lib.rs
2	programs/game/src/constants.rs
3	programs/game/src/errors.rs
4	programs/game/src/events.rs
5	programs/game/src/state.rs
6	programs/game/src/state/tests.rs
7	programs/game/src/utils/mod.rs
8	programs/game/src/utils/pda.rs
9	programs/game/src/utils/signature.rs
10	programs/game/src/utils/ids.rs
11	programs/game/src/utils/vrf_helpers.rs
12	programs/game/src/utils/fee_vault.rs
13	programs/game/src/utils/emission.rs
14	programs/game/src/utils/marketplace_fee.rs
15	programs/game/src/utils/token.rs
16	programs/game/src/instructions/mod.rs
17	programs/game/src/instructions/utility/mod.rs
18	programs/game/src/instructions/utility/estimate.rs
19	programs/game/src/instructions/game/mod.rs
20	programs/game/src/instructions/game/rewards.rs
21	programs/game/src/instructions/game/farm.rs
22	programs/game/src/instructions/game/join.rs
23	programs/game/src/instructions/admin/mod.rs
24	programs/game/src/instructions/admin/initialize.rs
25	programs/game/src/instructions/admin/config.rs

Id	Files in scope
26	programs/game/src/instructions/admin/pause.rs
27	programs/game/src/instructions/admin/fee_vault.rs
28	programs/game/src/instructions/admin/emergency.rs
29	programs/game/src/instructions/marketplace/mod.rs
30	programs/game/src/instructions/marketplace/listing_validation.rs
31	programs/game/src/instructions/marketplace/sale.rs
32	programs/game/src/instructions/marketplace/sale/common.rs
33	programs/game/src/instructions/marketplace/sale/create.rs
34	programs/game/src/instructions/marketplace/sale/buy.rs
35	programs/game/src/instructions/marketplace/sale/cancel.rs
36	programs/game/src/instructions/breeding/mod.rs
37	programs/game/src/instructions/breeding/request.rs
38	programs/game/src/instructions/breeding/vrf.rs
39	programs/game/src/instructions/breeding/resolve.rs
40	programs/game/src/instructions/breeding/pack_opening.rs
41	programs/game/src/instructions/breeding/abandon.rs
42	programs/game/src/instructions/breeding/attempt_cleanup.rs

Roles

Id	Roles
1	Admin
2	Signature Authority
3	Treasury
4	User

Executive Summary

Issues Found

Severity	Count	Description
High	3	Critical vulnerabilities
Medium	3	Significant risks
Low	5	Minor issues with low impact
Informational	0	Best practices or suggestions
Gas	1	Optimization opportunities

Issues Statuses

Id	Title	Status
H-01	Rogue Config Admin Can Operate On Shared Assets	Fixed
H-02	‘resolve_breeding‘ Does Not Verify Parent Ownership	Fixed
H-03	‘admin_shutdown_close_prisoner‘ Closes A Farming Prisoner Without Decrementing GP Accumulators Or Cleaning The Placement Array	Acknowledged
M-01	Randomness-Ready Breeding Attempts Can Be Abandoned After Outcome Is Known	Fixed
M-02	Pack Opening Has No Non-Destructive Recovery If VRF Never Fulfills	Fixed
M-03	Retroactive Marketplace Fee Increase Drains All Seller Proceeds	Fixed
L-01	Admin Can Lower ‘max_prisoner_gp‘ Below Existing Template GPs And Break Pack Opens	Fixed
L-02	Admin Can Create A Prisoner With Uncapped GP To Monopolize All Farming Emissions	Fixed
L-03	SA Signature Replay On ‘resolve_breeding‘ / ‘resolve_pack_opening‘	Fixed

Id	Title	Status
L-04	JAIL Token ‘maximum_fee = u64::MAX‘ With Admin-Held Transfer Fee Authority	Fixed
L-05	SA Can Sign ‘template_id‘ 21-255 Producing A Zero-GP Prisoner	Fixed
G-01	Marketplace Buy Performs A Zero-lamport Seller Transfer When Fee Rate Is 100%	Fixed

Security Grade



Based on the audit findings and code quality, the overall protocol security grade is 83.00 / 100.00. This reflects a good security posture with small room for improvement.

Findings

High

High 01 Rogue Config Admin Can Operate On Shared Assets

Location

rewards.rs:17

Description

`PlayerState`, `Prisoner`, `Pack`, `BreedingAttempt`, `ListingCounter`, and `SaleListing` are globally derived without `config_seed`, while `GameConfig`, `GlobalStats`, the JAIL mint, and mint authority are scoped by `config_seed`.

One concrete example of this vulnerability:

Because `initialize_system` is permissionless for any unused `config_seed`, an attacker can initialize a rogue config where they are both admin and signature authority. They can then authorize their own `join_game` under that rogue config. The join writes to the global `PlayerState` PDA because the player state seed does not include `config_seed`.

After that, the attacker can call `claim_rewards` using the official config. `claim_rewards` accepts the same global `PlayerState` PDA but derives the official `GlobalStats`, official JAIL mint, and official mint authority from the official config. There is no validation tying the `PlayerState` to the config under which it was created.

Attack pattern:

1. Attacker initializes rogue config with attacker-controlled signature authority.
2. Attacker signs their own join under the rogue config.
3. Rogue join creates or controls the global `PlayerState`.
4. Attacker calls `claim_rewards` against the official config.
5. Official JAIL can be minted using attacker-controlled grow power that was not created under the official config/accounting domain.

Proof of Concept

Put this test:

```rust
#[cfg(test)]
mod tests {
    use super::*;

    fn sample_config(
        config_seed: [u8; 32],
        admin: Pubkey,
        signature_authority: Pubkey,
    ) -> GameConfig {
        GameConfig {
            config_seed,
            bump: 0,
            admin,
            treasury: Pubkey::new_unique(),
            farm_upgrade_costs: FARM_UPGRADE_COSTS,
            signature_authority,
            emission_table: DEFAULT_EMISSION_TABLE,
            emission_terminal: DEFAULT_EMISSION_TERMINAL,
            paused_functions: 0,
            game_start_time: 0,
            marketplace_fee_rate: DEFAULT_MARKETPLACE_FEE_RATE,
            max_prisoner_gp: DEFAULT_MAX_PRISONER_GP,
            prisoner_template_gps: DEFAULT_PRISONER_TEMPLATE_GPS,
            join_fee: DEFAULT_JOIN_FEE,
            bootstrapped: false,
        }
    }

    #[test]
    fn rogue_config_joined_player_can_claim_inflated_official_rewards() {
        let program_id = crate::id();
        let official_config_seed = [31u8; 32];
        let rogue_config_seed = [41u8; 32];
        let official_admin = Pubkey::new_unique();
        let official_signature_authority = Pubkey::new_unique();
        let attacker = Pubkey::new_unique();

        let official_config =
            sample_config(official_config_seed, official_admin,
    official_signature_authority);
        let mut rogue_config = sample_config(rogue_config_seed, attacker, attacker);
        rogue_config.join_fee = 0;
        rogue_config.max_prisoner_gp = DEFAULT_MAX_PRISONER_GP;

        let (official_config_pda, _) =
            Pubkey::find_program_address(&[CONFIG_SEED, &official_config_seed], &
    program_id);
```

```
        let (rogue_config_pda, _) =
            Pubkey::find_program_address(&[CONFIG_SEED, &rogue_config_seed], &program_id
);
        let (official_jail_mint, _) =
            Pubkey::find_program_address(&[JAIL_MINT_SEED, &official_config_seed], &
program_id);
        let (rogue_jail_mint, _) =
            Pubkey::find_program_address(&[JAIL_MINT_SEED, &rogue_config_seed], &
program_id);

        assert_ne!(official_config_pda, rogue_config_pda);
        assert_ne!(official_jail_mint, rogue_jail_mint);

        let (official_view_of_attacker_state, _) =
            Pubkey::find_program_address(&[PLAYER_SEED, attacker.as_ref()], &program_id)
;
        let (rogue_view_of_attacker_state, _) =
            Pubkey::find_program_address(&[PLAYER_SEED, attacker.as_ref()], &program_id)
;

        assert_eq!(official_view_of_attacker_state, rogue_view_of_attacker_state);

        assert_eq!(rogue_config.admin, attacker);
        assert_eq!(rogue_config.signature_authority, attacker);

        let rogue_signed_join_gp = u64::from(rogue_config.max_prisoner_gp) * 2;
        let mut attacker_state_created_by_rogue_join = PlayerState {
            bump: 0,
            total_grow_power: rogue_signed_join_gp,
            last_harvest_time: 0,
            pending_farming_rewards: 0,
            reward_per_grow_power_paid: 0,
            pending_reward_remainder: 0,
            farm_level: 1,
            placed_prisoner_ids: [0; MAX_PLACED_PRISONERS],
            placed_count: INITIAL_PRISONER_COUNT,
            last_upgrade_time: 0,
            last_nonce: 1,
        };

        let official_global_stats = GlobalStats {
            bump: 0,
            total_grow_power: 250,
            acc_reward_per_grow_power: 0,
            reward_remainder: 0,
            last_reward_time: 0,
        };
```

```
88
89          let minted_by_official_claim = emission::accumulate_pending_rewards(
90              &mut attacker_state_created_by_rogue_join,
91              &official_global_stats,
92              &official_config,
93              SECONDS_PER_DAY,
94          )
95          .unwrap();
96
97          assert_eq!(
98              attacker_state_created_by_rogue_join.pending_farming_rewards,
99              minted_by_official_claim
100         );
101         assert_eq!(
102             minted_by_official_claim,
103             official_config.emission_table[0] * 8_800
104         );
105         assert!(minted_by_official_claim > official_config.emission_table[0]);
106     }
107 }
```

at the bottom of `programs/game/src/instructions/game/rewards.rs`.

Run it with:

```
1 cargo test -p game rogue_config_joined_player_can_claim_inflated_official_rewards
```

This demonstrates concrete impact: the attacker-controlled rogue join leads to an official reward claim worth `8,800x` the official day-0 emission in the modeled setup.

Recommendation

Scope every game-owned user/asset PDA to `config_seed`, including `PlayerState`, `Prisoner`, `Pack`, `BreedingAttempt`, `ListingCounter`, and `SaleListing`.

For existing accounts, add an explicit `config_seed` or `game_config` field to the account data and validate it in every instruction that mixes config-scoped and currently-global accounts. `ClaimRewards` should reject any `PlayerState` that was not created for the supplied `GameConfig`.

Status

Fixed

High 02 `resolve_breeding` Does Not Verify Parent Ownership

Location

resolve.rs:26-40

request.rs:186-263

Description

`request_breeding` correctly validates `parent.owner == player` at the time of request, but it does not lock the parents — their status remains IDLE or FARMING throughout the breeding window. `resolve_breeding` then checks only `parent.id == breeding_attempt.parent1_id` with no owner constraint:

```
// resolve.rs:26-39
#[account(
    seeds = [PRISONER_SEED, &parent1_prisoner.id.to_le_bytes()],
    bump = parent1_prisoner.bump,
    constraint = parent1_prisoner.id == breeding_attempt.parent1_id @ GameError::
    ParentMismatch,
    // ← no constraint: parent1_prisoner.owner == player.key()
)]
pub parent1_prisoner: Box<Account<'info, Prisoner>>,
```

Because parents remain IDLE after `request_breeding`, the owner can immediately list a parent on the marketplace. A sale completes, transferring ownership to a buyer. When the original requester later calls `resolve_breeding`, the constraint passes (id still matches), and the breeding output (1-2 packs) is credited to the original requester. The buyer's prisoner has its `breeding_count` incremented and `last_bred_at` set without consent.

The impact is the following:

- Original requester receives both the sale proceeds for the parent AND the breeding output.
- New owner (buyer) has one of their seven lifetime breeding slots permanently consumed without authorization. There is no on-chain function for the buyer to recover this slot.
- When combined with H-02 (SA can sign zero breeding cost), the original requester pays nothing, sells the parent, and receives packs — a complete economic exploit with no cost.
- Buyer is permanently locked into a corrupted state; `abandon_breeding_attempt` is only available to the original requester, and only before resolution.

Recommendation

Option A (preferred): Add owner constraint at resolution:

```
constraint = parent1_prisoner.owner == player.key() @ GameError::NotPrisonerOwner,
```

Option B: Lock parents at request time by setting a new `STATUS_BREEDING_LOCKED` status, and unlock at resolve or abandon. This prevents marketplace listing during the breeding window.

Status

Fixed

High 03 `admin_shutdown_close_prisoner` Closes A Farming Prisoner Without Decrementing GP Accumulators Or Cleaning The Placement Array

Location

emergency.rs:313-355

Description

The `AdminShutdownClosePrisoner` account struct accepts only three accounts — `admin`, `prisoner`, and `game_config` — and contains no reference to `PlayerState` or `GlobalStats`. As a result, when the instruction closes a prisoner whose status is `STATUS_FARMING` (status value `1`), neither `PlayerState.total_grow_power` nor `GlobalStats.total_grow_power` is decremented, and the prisoner ID is never removed from `PlayerState.placed_prisoner_ids`. The only status blocked by a constraint is `STATUS_SALE_LISTED` (value `2`); farming prisoners pass unchecked:

```
// emergency.rs:316-338
pub struct AdminShutdownClosePrisoner<'info> {
    pub admin: Signer<'info>,
    pub prisoner: Account<'info, Prisoner>,   // close = admin
    pub game_config: Box<Account<'info, GameConfig>>,
    // NO player_state, NO global_stats
}

// emergency.rs:328
```

```
10 constraint = prisoner.status ≠ Prisoner::STATUS_SALE_LISTED @ GameError::
       InvalidPrisonerStatus,
```

After the prisoner PDA is closed (lamports returned to admin), the ghost GP remains alive in both accumulators indefinitely. The inverse decrement path in `batch_update_farm` requires the prisoner PDA to exist and hold `STATUS_FARMING`; with the PDA zeroed, the decrement is permanently unreachable.

The impact is the following:

Any honest farmer's JAIL emission share is permanently diluted in proportion to the ghost GP versus total GP. Because `GlobalStats.total_grow_power` is the shared denominator in the reward accumulator formula, the impact applies to every farmer simultaneously — not just those who interact with the affected player. The ghost GP persists across program upgrades unless a dedicated repair instruction is added. Per-player `placed_prisoner_ids` also retains a stale entry, causing an off-by-one in placement slot accounting.

Recommendation

Add `player_state` and `global_stats` as mutable accounts in `AdminShutdownClosePrisoner`. Before closing the prisoner, check its status and, if `STATUS_FARMING`, subtract `prisoner.gp` from both `player_state.total_grow_power` and `global_stats.total_grow_power`, call `remove_placed_prisoner(prisoner_id)` on the `PlayerState`, and decrement `placed_count`.

```
// Proposed fix in AdminShutdownClosePrisoner:
pub player_state: Account<'info, PlayerState>,
pub global_stats: Account<'info, GlobalStats>,

// In handler, before PDA close:
if prisoner.status == Prisoner::STATUS_FARMING {
    player_state.total_grow_power = player_state.total_grow_power
        .checked_sub(u64::from(prisoner.gp))
        .ok_or(GameError::GrowPowerArithmetic)?;
    global_stats.total_grow_power = global_stats.total_grow_power
        .checked_sub(u64::from(prisoner.gp))
        .ok_or(GameError::GrowPowerArithmetic)?;
    player_state.remove_placed_prisoner(prisoner.id)?;
}
```

Status

Acknowledged

Medium

Mid 01 Randomness-Ready Breeding Attempts Can Be Abandoned After Outcome Is Known

Location

attempt_cleanup.rs:27

Description

`validate_reclaimable_attempt` accepts both `AwaitingRandomness` and `RandomnessReady`. Once an attempt is `RandomnessReady`, the VRF randomness is already stored on-chain and observable. The user can derive the outcome, abandon bad rolls after the timeout, and restore parent breeding count/cooldown.

Proof of Concept

Put this test:

```
#[test]
fn randomness_ready_attempt_can_be_abandoned_after_outcome_is_known() {
    let requested_at = 10_000;
    let now = requested_at + ADMIN_CLOSE_ATTEMPT_MIN_AGE;
    let mut attempt = sample_attempt(BreedingAttempt::STATUS_RANDOMNESS_READY,
    requested_at);
    attempt.randomness = Some([7u8; 32]);
    let mut parent1 = sample_prisoner(Prisoner::STATUS_FARMING, 4, requested_at);
    let mut parent2 = sample_prisoner(Prisoner::STATUS_IDLE, 5, requested_at);

    assert!(attempt.randomness.is_some());
    assert_eq!(attempt.status, BreedingAttempt::STATUS_RANDOMNESS_READY);

    assert_eq!(
        validate_reclaimable_attempt(&attempt, now).unwrap(),
        ADMIN_CLOSE_ATTEMPT_MIN_AGE
    );

    restore_parent_after_unresolved_attempt(&mut parent1, attempt.requested_at);
    restore_parent_after_unresolved_attempt(&mut parent2, attempt.requested_at);

    assert_eq!(parent1.breeding_count, 3);
    assert_eq!(parent2.breeding_count, 4);
```

```
23      assert_eq!(parent1.last_bred_at, 0);
24      assert_eq!(parent2.last_bred_at, 0);
25  }
```

inside `programs/game/src/instructions/breeding/attempt_cleanup.rs`, in the existing `#[cfg(test)] mod tests`.

Run:

```
1  cargo test -p game randomness_ready_attempt_can_be_abandoned_after_outcome_is_known
```

Recommendation

Do not allow user abandon after randomness is ready. If ready attempts need recovery, require objective unresolvability and do not restore parent breeding count/cooldown after randomness has been revealed.

Status

Fixed

Mid 02 Pack Opening Has No Non-Destructive Recovery If VRF Never Fulfills

Location

pack_opening.rs:47

Description

`request_pack_opening` moves a pack from `Idle` to `Opening`. From there, only the VRF callback can move it to `ReadyToOpen`. If the callback never arrives, the user cannot request opening again, cannot resolve, and cannot list the pack for sale.

Proof of Concept

Put this test:

```rust
#[cfg(test)]
mod tests {
    use super::*;

    fn sample_pack(status: u8) -> Pack {
        Pack {
            id: 42,
            bump: 0,
            owner: Pubkey::new_unique(),
            status,
            parent1_id: 1,
            parent2_id: 2,
            rarity: 0,
            breeding_randomness: [3u8; 32],
            open_randomness: None,
            created_at: 1_000,
            open_requested_at: 0,
        }
    }

    fn user_can_request_opening(pack: &Pack) -> bool {
        pack.status == Pack::STATUS_IDLE
    }

    fn user_can_resolve_opening(pack: &Pack) -> bool {
        pack.status == Pack::STATUS_READY_TO_OPEN && pack.open_randomness.is_some()
    }

    fn user_can_list_pack_for_sale(pack: &Pack) -> bool {
        pack.status == Pack::STATUS_IDLE
    }

    #[test]
    fn pack_opening_request_leaves_pack_stuck_without_vrf_callback() {
        let mut pack = sample_pack(Pack::STATUS_IDLE);
        let requested_at = 2_000;

        assert!(user_can_request_opening(&pack));
        pack.status = Pack::STATUS_OPENING;
        pack.open_randomness = None;
        pack.open_requested_at = requested_at;

        assert!(!user_can_request_opening(&pack));
        assert!(!user_can_resolve_opening(&pack));
        assert!(!user_can_list_pack_for_sale(&pack));

        assert_eq!(pack.status, Pack::STATUS_OPENING);
```

```
        assert_eq!(pack.open_requested_at, requested_at);
        assert!(pack.open_randomness.is_none());
    }

    #[test]
    fn only_vrf_fulfillment_makes_stuck_pack_resolvable() {
        let mut pack = sample_pack(Pack::STATUS_OPENING);
        pack.open_requested_at = 2_000;

        assert!(!user_can_resolve_opening(&pack));

        pack.open_randomness = Some([9u8; 32]);
        pack.status = Pack::STATUS_READY_TO_OPEN;

        assert!(user_can_resolve_opening(&pack));
    }
}
```

at the bottom of `programs/game/src/instructions/breeding/pack_opening.rs`.

Run:

```
cargo test -p game pack_opening_request_leaves_pack_stuck_without_vrf_callback
cargo test -p game only_vrf_fulfillment_makes_stuck_pack_resolvable
```

Recommendation

Add a stale-opening recovery instruction, such as `cancel_pack_opening` or `retry_pack_opening`, gated by `open_requested_at + timeout`.

Status

Fixed

Mid 03 Retroactive Marketplace Fee Increase Drains All Seller Proceeds

Location

buy.rs:92

buy.rs:186

Description

Both `handle_buy_listing` (prisoner sales) and `handle_buy_pack_listing` (pack sales) compute the fee split by reading `marketplace_fee_rate` live from `GameConfig` at the moment of purchase. The `SaleListing` account does not record the fee rate that was in effect when the listing was created. This means the admin can raise the fee rate to `FEE_DENOMINATOR` (10,000 — representing 100%) at any time after sellers have created their listings, and every subsequent purchase will route the entire sale price to the fee vault, leaving sellers with nothing.

```
// buy.rs:86-93 — prisoner listing purchase
let fee = transfer_marketplace_sale_payment(
    ctx.accounts.buyer.to_account_info(),
    ctx.accounts.seller.to_account_info(),
    ctx.accounts.fee_vault_shard.to_account_info(),
    ctx.accounts.system_program.to_account_info(),
    details.price,
    ctx.accounts.game_config.marketplace_fee_rate,  // live read — not snapshotted at
    listing time
)?;

// buy.rs:180-187 — pack listing purchase (identical pattern)
let fee = transfer_marketplace_sale_payment(
    ctx.accounts.buyer.to_account_info(),
    ctx.accounts.seller.to_account_info(),
    ctx.accounts.fee_vault_shard.to_account_info(),
    ctx.accounts.system_program.to_account_info(),
    details.price,
    ctx.accounts.game_config.marketplace_fee_rate,  // same live read
)?;
```

The `SaleListing` account holds only `{ seller, asset_type, asset_id, price }` — there is no `fee_rate` field:

```
// state.rs:315-321
pub struct SaleListing {
    pub bump: u8,
    pub seller: Pubkey,
    pub asset_type: u8,
    pub asset_id: u32,
    pub price: u64,       // listing price only — fee rate not captured at creation
}
```

The admin setter enforces only an upper bound of `FEE_DENOMINATOR` (10,000), with no lower bound:

```
// config.rs:276-284
if let Some(v) = marketplace_fee_rate {
    require!(
        v ⩽ u16::try_from(crate::constants::FEE_DENOMINATOR)
            .expect("fee denominator fits in u16"),
        crate::errors::GameError::InvalidParameter
    );
    config.marketplace_fee_rate = v;
}
```

Setting `v = 10_000` is explicitly valid. The fee split function at `marketplace_fee.rs:28-40` is arithmetically correct — it produces exactly zero for the seller at this rate:

```
// marketplace_fee.rs:28-40
pub(crate) fn split_marketplace_fee(price: u64, fee_rate: u16) -> Result<(u64, u64)> {
    require!(u64::from(fee_rate) ⩽ FEE_DENOMINATOR, GameError::InvalidParameter);
    let fee_u128 = u128::from(price)
        .checked_mul(u128::from(fee_rate))
        .ok_or(GameError::InvalidParameter)?
        / u128::from(FEE_DENOMINATOR);
    let fee = u64::try_from(fee_u128).map_err(|_| GameError::InvalidParameter)?;
    let recipient_amount = price.checked_sub(fee).ok_or(GameError::InvalidParameter)?;
    Ok((fee, recipient_amount))
    // At fee_rate = 10_000: fee = price; recipient_amount = 0
}
```

This affects both prisoner listings and pack listings. Both `handle_buy_listing` and `handle_buy_pack_listing` share the same `transfer_marketplace_sale_payment` helper and the same live-read pattern.

The impact is the following:

- Direct fund loss for all active sellers: A seller who listed a prisoner at 10 SOL (10,000,000,000 lamports) expecting to receive 9 SOL after a 10% fee receives 0 lamports. The full 10 SOL goes to the protocol fee vault.
- Both listing types affected simultaneously: All prisoner listings and all pack listings are drained by a single `update_marketplace_config` call. No per-listing protection exists.

Recommendation

Snapshot the fee rate into `SaleListing` at creation time and use it at purchase time. This is the standard marketplace design: sellers see and agree to the fee at listing creation, and that rate is locked for the life of the listing.

```rust
// state.rs — add to SaleListing
pub struct SaleListing {
    pub bump: u8,
    pub seller: Pubkey,
    pub asset_type: u8,
    pub asset_id: u32,
    pub price: u64,
    pub fee_rate_at_listing: u16,  // ADD: snapshot current marketplace_fee_rate
}
```

```rust
// buy.rs — use snapshotted rate instead of live config read
let fee = transfer_marketplace_sale_payment(
    // ...
    sale_listing.fee_rate_at_listing,  // use stored rate, not game_config.
    marketplace_fee_rate
)?;
```

Additionally, enforce a maximum fee rate cap below 100% (e.g., 2,000 bps / 20%) so the setter cannot set a confiscatory rate even retroactively:

```rust
// config.rs
const MAX_MARKETPLACE_FEE_RATE: u16 = 2_000; // 20% ceiling
require!(v ⩽ MAX_MARKETPLACE_FEE_RATE, GameError::InvalidParameter);
```

Status

Fixed

Low

Low 01 Admin Can Lower `max_prisoner_gp` Below Existing Template GPs And Break Pack Opens

Location

config.rs:198

Description

`handle_update_game_config` lets the admin update `max_prisoner_gp` directly without checking it against the existing configured prisoner template GP table.

This matters because `resolve_pack_opening` reads the prisoner GP from `GameConfig.prisoner_template_gps[template_id]` and then enforces:

```
gp ⩽ game_config.max_prisoner_gp
```

Therefore, if the admin lowers `max_prisoner_gp` below one or more already-configured nonzero template GP values, packs resolving to those templates will fail during `resolve_pack_opening`.

This is admin-controlled and does not let an external attacker steal funds or bypass ownership. The issue is an operational/configuration footgun: a valid template table can become temporarily unusable because the cap is mutable independently from the template values that are checked against it.

Recommendation

When updating `max_prisoner_gp`, validate that the new value is greater than or equal to every existing nonzero entry in `game_config.prisoner_template_gps`.

Alternatively, document this behavior explicitly as an intentional emergency brake and require operational tooling to warn before lowering the cap below configured template GP values.

Status

Fixed

Low 02 Admin Can Create A Prisoner With Uncapped GP To Monopolize All Farming Emissions

Location

emergency.rs:98-99

Description

`admin_create_prisoner` creates a `Prisoner` account with a caller-supplied `gp` value of type `u32`. All other prisoner creation paths — `join_game` and `resolve_pack_opening` — enforce `require!(gp ⩽ config.max_prisoner_gp)`. The admin path explicitly skips this check, as noted by a comment in the source:

```
// emergency.rs:98-112
// No GP cap check — admin can set any GP
let prisoner = Prisoner {
    id: prisoner_id,
    bump: ctx.bumps.prisoner,
    owner,
    gp,           // caller-supplied u32, no require!(gp ⩽ config.max_prisoner_gp)
    rarity,
    template_id,
    status: Prisoner::STATUS_IDLE,
    parent1_id,
    parent2_id,
    breeding_count: 0,
    last_bred_at: 0,
};
```

The `Prisoner.gp` field is typed `u32` (maximum value 4,294,967,295). When this prisoner is placed in the farm via `batch_update_farm`, its GP is added to `global_stats.total_grow_power`:

```
// farm.rs:197
gp_placed += u64::from(prisoner_data.gp);  // u64::from(u32::MAX) = 4_294_967_295
```

The emission accumulator distributes rewards proportionally to each prisoner's share of total GP. All arithmetic uses `u128`, so there is no overflow — the attack is arithmetically clean:

```
// emission.rs:139-141
let total_gp = u128::from(global_stats.total_grow_power);
let acc_delta = numerator / total_gp;
let reward_remainder = numerator % total_gp;
```

With `u32::MAX` GP added to a pool where all legitimate players collectively hold approximately 100,000,000 GP:

```
admin_share = 4_294_967_295 / (4_294_967_295 + 100_000_000)
            = 4_294_967_295 / 4_394_967_295
            ≈ 97.73%
```

Every second of farming, the admin-controlled prisoner captures approximately 97.73% of all JAIL emission. At the peak Day-10 emission rate of 4,840,315,467,000,000 base units per day, the admin prisoner accumulates approximately 4,730,000,000,000,000 JAIL base units per day. All other players combined receive approximately 110,000,000,000,000 units per day — about 2.27% of the total.

The config setter for `max_prisoner_gp` has no lower-bound guard, enabling a compounding attack variant:

```
// config.rs:198-199
if let Some(v) = max_prisoner_gp {
    config.max_prisoner_gp = v;    // no require!(v > 0) — can be set to 0
}
```

Setting `max_prisoner_gp = 0` blocks all normal prisoner creation (join_game and resolve_pack_opening both check `gp ⩽ max_prisoner_gp`) while leaving `admin_create_prisoner` unaffected. This allows the admin to simultaneously monopolize emissions and block all new player participation.

The impact is the following:

- Permanent destruction of farming economics: All legitimate players' farming returns are diluted to approximately 2.27% of the emission schedule for as long as the monopoly prisoner remains in the farm. There is no on-chain mechanism to force-remove the prisoner; only the admin (owner) can remove it.
- Scale with participation: The more players participate, the larger the absolute value of emissions the admin captures. The 97.73% figure assumes 100M combined player GP; a real deployment with higher participation still leaves players at roughly the same fraction.
- Single transaction attack: One call to `admin_create_prisoner(gp=4_294_967_295)` followed by one `batch_update_farm` call completes the attack. No further interaction is required.
- Compound with terminal zeroing: Combined with M-03, admin can capture most of the season's rewards and then eliminate all remaining reward value for other players.

Recommendation

Apply the same `max_prisoner_gp` cap check in `admin_create_prisoner` that is already enforced in all other prisoner creation paths. The admin-exempt path is unnecessary for the function's stated purpose (emergency prisoner creation):

```
// emergency.rs — add before the Prisoner struct initialization
require!(
    gp ⩽ config.max_prisoner_gp,
    GameError::InvalidParameter
);
// No GP cap check — admin can set any GP  ← remove this comment and the unchecked
    assignment
```

Additionally, enforce a minimum value for `max_prisoner_gp` in the config setter to prevent the compounding `max_prisoner_gp=0` DoS variant:

```
// config.rs:198-199
if let Some(v) = max_prisoner_gp {
    require!(v > 0, GameError::InvalidParameter);  // ADD: prevent zero freeze
    config.max_prisoner_gp = v;
}
```

Status

Fixed

Low 03 SA Signature Replay On `resolve_breeding` / `resolve_pack_opening`

Location

signature.rs:198-200

resolve.rs:73

pack_opening.rs:146

Description

`resolve_breeding` and `resolve_pack_opening` authenticate the SA signature using `verify_expiry` only — no nonce is consumed. The `attempt_id` in `request_breeding` and `pack_id` in `resolve_pack_opening` are player-supplied `u32` values with no global counter enforced on-chain. After a `BreedingAttempt` or `Pack` account is closed (on resolution or abandonment), the same ID may be reused for a new breeding request. An SA-signed resolve message for the old attempt — binding `(attempt_id, parent1_id, parent2_id, rarity, pack_count, expires_at)` — remains valid for the new attempt if the ID and parent IDs coincidentally match and the signature has not yet expired.

```
// resolve.rs:73 — expiry only, no nonce
signature::verify_expiry(expires_at, now)?;

// pack_opening.rs:146 — same pattern
signature::verify_expiry(expires_at, now)?;

// signature.rs:198-200 — no upper bound on expires_at
pub fn verify_expiry(expires_at: i64, now: i64) -> Result<()> {
    require!(expires_at > now, GameError::SignatureExpired);
```

```
10         Ok(())
11     }
```

This is a self-exploitation vector only: a player who intentionally reuses an `attempt_id` within the SA expiry window and matches the same parent IDs can force a prior rarity or pack count onto the new attempt. The practical window is the SA-enforced backend expiry (expected 600 seconds per recon), but this is not enforced on-chain (see L-07). If the SA system ever issues long-expiry resolve signatures, the window is much wider.

The impact is the following:

A player can selectively replay a favorable resolve outcome (specific rarity, pack count) on a newly created attempt with the same ID, provided the archived SA signature for the old attempt is unexpired. No third-party theft occurs; impact is confined to the individual player manipulating their own breed results.

Recommendation

Add a nonce parameter to both `resolve_breeding` and `resolve_pack_opening` messages and consume `player_state.last_nonce` during resolution, analogous to `request_breeding`. As a partial mitigation, enforcing a short maximum expiry (see L-07 recommendation) reduces the practical replay window to the bounded SA validity period.

Status

Fixed

Low 04 JAIL Token `maximum_fee = u64::MAX` With Admin-Held Transfer Fee Authority

Location

token.rs:120

constants.rs:21

Description

The JAIL Token-2022 mint is initialized with `maximum_fee = u64::MAX` and a default transfer fee rate of 200 bps (2%). The `maximum_fee` field in the Token-2022 `TransferFeeConfig` extension sets an absolute per-transfer cap in token base units; setting it to `u64::MAX` means there is effectively no

per-transfer ceiling on withheld fees. The admin holds both `transfer_fee_config_authority` and `withdraw_withheld_authority` off-program — no program-level instruction controls these authorities.

```
// constants.rs:21
pub const TRANSFER_FEE_BPS: u16 = 200; // 2%

// token.rs:120 — no practical per-transfer cap
maximum_fee: u64::MAX,
```

At the default 2% rate, the uncapped `maximum_fee` has limited immediate impact since 2% of any reasonable transfer is well below `u64::MAX`. However, the admin can raise `transfer_fee_basis_points` to 10,000 (100%) via an external Token-2022 instruction (see M-01) and simultaneously benefit from `maximum_fee = u64::MAX` — all withheld amounts from any transfer are unbounded. For ecosystem participants making large JAIL transfers (e.g., OTC, DEX liquidity), the absence of a per-transfer cap means unexpected fee amounts if the rate is ever raised.

The impact is the following:

External JAIL holders and ecosystem protocols cannot bound their worst-case transfer cost: with `maximum_fee = u64::MAX`, any raise in transfer fee rate up to 100% takes the full transfer amount. There is no on-chain guarantee of a per-transfer ceiling.

Recommendation

Set `maximum_fee` to a reasonable absolute value based on expected transfer sizes and game economics — for example, capping it at 1,000,000 base units (1 JAIL with 6 decimals) per transfer prevents large disproportionate fee extractions even if the rate is temporarily raised. Document the chosen cap and its rationale in the codebase.

Status

Fixed

Low 05 SA Can Sign `template_id` 21-255 Producing A Zero-GP Prisoner

Location

pack_opening.rs:147-154

constants.rs:49-73

Description

The `DEFAULT_PRISONER_TEMPLATE_GPS` array has 256 entries; only indices 1-20 are non-zero:

```
pub const DEFAULT_PRISONER_TEMPLATE_GPS: [u32; 256] = {
    let mut gps = [0; 256];
    gps[1] = 125;
    // ... gps[2..20] populated
    gps[20] = 1_100_000;
    gps  // gps[21..=255] all remain 0
};
```

The only on-chain check is `template_id ≠ 0`. There is no upper-bound check and no `require!(gp > 0)` after fetching the GP value:

```
ids::require_nonzero_prisoner_template_id(template_id)?;  // rejects 0 only
let gp = config.prisoner_template_gp(template_id)?;       // returns 0 for template_id 21-255
require!(gp ⩽ config.max_prisoner_gp, GameError::GpExceedsMax);  // 0 ⩽ max → passes
```

The impact is the following:

SA signing `template_id = 21` through `255` produces a prisoner with `gp = 0`. This prisoner occupies a farm slot, contributes zero rewards, and cannot be distinguished by the contract from a legitimate prisoner. The player has spent breeding costs and parent breeding slots for a worthless asset with no recourse — `resolve_pack_opening` closes the Pack PDA (`close = player`), so the investment is consumed.

Recommendation

Either add an upper bound:

```
require!(template_id ⩽ MAX_VALID_TEMPLATE_ID, GameError::InvalidTemplateId);
```

where `MAX_VALID_TEMPLATE_ID = 20`, or add a post-fetch guard:

```
require!(gp > 0, GameError::InvalidTemplateId);
```

Status

Fixed

Gas

Gas 01 Marketplace Buy Performs A Zero-lamport Seller Transfer When Fee Rate Is 100%

Location

buy.rs:207

Description

`transfer_marketplace_sale_payment` always performs a system transfer from the buyer to the seller, even when the computed `seller_amount` is zero.

A zero seller amount is possible because `marketplace_fee_rate` is allowed to be `10_000`, meaning 100% of the listing price is taken as marketplace fee:

```
fee = price
```

```
seller_amount = 0
```

In that case, the seller transfer performs unnecessary CPI work before the fee is transferred to the fee vault. This is not a security issue and does not change accounting, but it wastes compute on marketplace buy transactions when the seller amount is zero.

Recommendation

Skip the seller transfer when `seller_amount == 0`, similar to the existing zero-amount guard used by `fee_vault::transfer_sol_fee_to_vault`.

Status

Fixed